

Script de soutenance — mot à mot (~20 min)

À lire à voix haute pour répéter, puis à t'approprier (n'apprends pas par cœur — retiens les idées). Indications entre crochets = gestes/posture, pas à dire. Vise un débit calme : ~140 mots/min.

Slide 1 — Titre ■ ~30 s

Bonjour à toutes et à tous. Je m'appelle Hischem Hammoudi, je suis alternant en deuxième année du cycle ingénieur à l'EFREI, majeure Big Data et Machine Learning, en poste chez Diagperf comme Data Analyst et automatisation. Je vais vous présenter mon mémoire, intitulé « De l'API au RAG » : il porte sur la conception d'un assistant conversationnel **fiable** pour la relation client d'une PME du secteur automobile. [regarder le jury, sourire] Je vous propose de commencer par le contexte.

Slide 2 — Contexte & enjeu ■ ~1 min 30

Diagperf est un garage spécialisé dans la reprogrammation moteur et le diagnostic électronique, situé à Villenoy, en Seine-et-Marne. C'est une petite structure : trois personnes. Les clients posent leurs questions surtout par téléphone — des demandes de devis, de rendez-vous, ou des questions techniques, par exemple sur la compatibilité à l'éthanol E85 ou sur un code défaut. Ça représente une à quatre demandes par jour : un volume modeste, mais un traitement très répétitif, et surtout à forte intensité de conseil. À chaque appel, il faut vérifier une compatibilité, annoncer un tarif exact, interpréter un symptôme. Et c'est là tout l'enjeu : dans ce métier, **une erreur n'est pas neutre**. Un prix faux, ou une compatibilité fausse, annoncés avec assurance, ça se traduit par une perte de confiance, parfois un litige. Donc l'objectif n'est pas seulement d'automatiser : c'est d'automatiser **sans jamais donner d'information fausse**. C'est ce qui va structurer tout le mémoire.

Slide 3 — Problématique & questions de recherche ■ ~1 min 30

Ce qui m'amène à ma problématique. [la lire posément] « Comment concevoir un assistant conversationnel fiable pour une PME automobile, capable d'automatiser le traitement des demandes clients, sans produire d'informations erronées ? » Je l'ai déclinée en trois questions de recherche. La première : comment traiter des demandes formulées en langage naturel, sans retomber dans la rigidité d'un système à boutons qui casse la conversation ? La deuxième, qui est le cœur scientifique : est-ce que le RAG, la génération augmentée par récupération, réduit vraiment les hallucinations par rapport à un modèle de langage utilisé par simple prompt ? Et la troisième, plus pratique : **où placer le savoir métier** — faut-il le figer dans le prompt, ou le mettre dans une base que l'on interroge ? On verra que cette troisième question donne un résultat assez contre-intuitif.

Slide 4 — Plan ■ ~20 s

Mon plan suit quatre temps : d'abord un état de l'art des approches possibles ; ensuite la conception de l'assistant, qui s'est faite en trois versions ; puis l'évaluation, avec un benchmark contrôlé ; et enfin la discussion, les limites et les perspectives.

Slide 5 — État de l'art ■ ~1 min 30

Commençons par situer les approches existantes. La première, ce sont les chatbots à règles, ou arbres de décision : c'est déterministe, prévisible, pas cher — mais c'est rigide, ça ne comprend pas le langage naturel, et la moindre question imprévue renvoie l'utilisateur au menu. La deuxième, c'est le modèle de langage utilisé par prompt, via une API : là, le modèle comprend enfin le langage — mais il hallucine, et son savoir est figé dans un long prompt. La troisième, c'est le RAG : au lieu d'enfermer les faits dans le modèle, on les met dans une base documentaire qu'on interroge à chaque question, et on ancre la réponse dans les passages récupérés. La quatrième, c'est le fine-tuning, comme LoRA : on modifie les poids du modèle — utile pour un style, mais ça exige beaucoup de données étiquetées, ce qu'une PME n'a pas. Et enfin les agents, qui peuvent agir, appeler des outils : je le garde comme perspective. [transition] Parmi tout ça, c'est le RAG qui répond le mieux à mon problème.

Slide 6 — Pourquoi le RAG pour une PME ■ ~1 min

Pourquoi le RAG, justement, pour une PME ? D'abord parce qu'il **sépare le savoir de la génération** : c'est le principe posé par Lewis et ses co-auteurs en 2020. Ensuite parce que cet ancrage documentaire réduit fortement les hallucinations — c'est démontré expérimentalement, par exemple par Shuster en 2021. Mais il y a deux bénéfices très concrets côté PME. La maintenabilité, d'abord : quand un tarif change, je n'ai qu'à éditer un fichier — pas besoin de réentraîner quoi que ce soit. Et l'adéquation aux données : Diagperf a peu de données, donc le fine-tuning est exclu, alors que le RAG fonctionne très bien avec une base documentaire modeste.

Slide 7 — La solution : un pipeline hybride ■ ~1 min

Voici l'architecture de la solution. Elle est **hybride**. Quand un message WhatsApp arrive, il est d'abord authentifié par une signature de sécurité. Ensuite, deux cas. Si on est dans une action en cours — produire un devis, prendre un rendez-vous — c'est une machine à états déterministe qui guide le client, étape par étape. Mais pour toute question libre, on passe au RAG et au modèle de langage. Et un point important : le modèle ne renvoie jamais du texte brut. Il renvoie un objet structuré, du JSON, qui dit soit « voici la réponse », soit « rebascule vers tel flow ». Ça me permet de combiner la souplesse de la conversation avec le **contrôle des actions sensibles**, comme la création d'un devis.

Slide 8 — Vue client : le parcours nominal ■ ~45 s

Concrètement, voici ce que voit le client. [montrer l'écran] À gauche, l'accueil : un message de bienvenue et un bouton « Nos prestations ». À droite, la liste des prestations — reprogrammation, conversion E85, suppressions FAP, EGR, AdBlue, diagnostic, et le SAV. Tout ça, ce sont des **messages interactifs natifs de WhatsApp** : des boutons, des listes. Le parcours est guidé, le client n'a pas besoin de tout taper. Bref : le produit fonctionne. Maintenant, je vais vous montrer comment on en est arrivé là.

Slide 9 — Une conception en trois temps ■ ~1 min

La conception s'est faite de façon **itérative**, en trois versions que l'historique Git permet de dater précisément. La version 0, à la mi-avril : un simple arbre de décision, à boutons, sans intelligence artificielle. La version 1, début mai : un modèle de langage par prompt — il comprend enfin le langage, mais il invente les faits. Et la version 2, de mi-mai à début juin : l'architecture RAG. Ce qui compte, c'est que **chaque version corrige une limite concrète de la précédente**. Laissez-moi vous montrer précisément ce qui ne marchait pas en version 1 — parce que c'est ça qui justifie tout le reste.

Slide 10 — V1 : rupture de conversation ■ ~30 s

Premier cas, une vraie capture. Le client demande : « Est-ce que je peux rouler tranquillement ? » Une question parfaitement légitime. Et le bot répond : « Je n'ai pas bien saisi votre message » — et le renvoie au menu. La question libre n'est pas comprise : c'est une rupture de conversation.

Slide 11 — V1 : le flow à boutons rejette la question ■ ~30 s

Deuxième cas. Le client est en train de choisir un stage de reprogrammation, et il demande : « Est-ce que je risque quelque chose après ma reprog ? » Une question de réassurance, très normale. Et là, le bot répond : « Merci de choisir un des stages proposés ». Autrement dit, le système rigide rejette la question parce qu'elle ne rentre pas dans le script.

Slide 12 — V1 : la question prise pour une plaque ■ ~30 s

Troisième cas, le plus parlant. Le bot attend une plaque d'immatriculation. Le client, lui, pose une question de compatibilité : « Est-ce que je peux faire une reprog pour un véhicule essence ? » Et le bot répond : « Je n'ai pas reconnu la plaque ». La question a été prise pour une plaque illisible. [pause] Trois ruptures distinctes, sur trois états différents du système. C'est exactement cette fragilité qui m'a poussé vers le RAG.

Slide 13 — Architecture RAG (V2) ■ ~1 min 30

Voyons donc l'architecture RAG. Elle a deux temps. D'abord, une indexation **hors-ligne** : j'ai formalisé tout le savoir de Diagperf dans vingt-quatre fichiers, au format question-réponse — les tarifs, les compatibilités, la FAQ, les codes défauts. Chaque fichier est découpé en fragments, et chaque fragment est transformé en vecteur — un embedding — par l'API de Google, puis stocké dans une base Supabase avec l'extension pgvector. Ensuite, l'interrogation **en ligne** : quand une question arrive, j'enrichis d'abord la requête avec des synonymes, je la transforme en vecteur, et je fais une **recherche hybride** — c'est-à-dire que je combine la proximité sémantique, par les vecteurs, et la correspondance de mots exacts, par le plein-texte. C'est utile pour rattraper des termes précis comme un code défaut. Les meilleurs passages sont reclassés, puis injectés dans le prompt du modèle. Et c'est seulement à ce moment-là que Claude rédige la réponse — à partir de faits vérifiables, pas de sa mémoire.

Slide 14 — Côté technique : WhatsApp Business + backend Supabase ■ ~1 min

Un mot sur l'infrastructure, rapidement. Le canal, c'est la plateforme WhatsApp Business de Meta, l'API officielle : elle gère les messages interactifs — boutons, listes — le texte, les vocaux, et chaque message entrant est signé pour la sécurité. Le backend, c'est du Node.js avec Express, hébergé sur Render en déploiement continu. Et au centre, Supabase, ma base de données, qui a un **double rôle** intéressant : d'un côté elle stocke l'**état de la conversation** — où en est le client dans son parcours — et de l'autre elle héberge la **base de connaissances vectorielle** qui sert au RAG. Donc un seul backend orchestre tout : le canal, l'état, la recherche, et le modèle.

Slide 15 — Le benchmark : protocole ■ ~1 min 30

J'arrive à l'évaluation, qui est le cœur de ma contribution. Comme l'assistant n'est pas encore déployé, j'ai fait un **benchmark contrôlé hors-ligne**. J'ai constitué un jeu de trente et une questions : vingt-huit ancrées sur la base, et trois cas réels, transcrits des bugs que je viens de vous montrer. Et surtout, j'ai passé chaque question dans **quatre conditions**, en croisant deux facteurs : le prompt — avec ou sans le savoir baké dedans — et le RAG — activé ou non. L'intérêt, c'est l'**ablation** : en comparant la même configuration avec et sans RAG, j'**isole l'effet causal du RAG**, toutes choses égales par ailleurs. Pour la notation, j'utilise un second modèle comme juge — Claude Opus, plus puissant que le modèle évalué — une méthode établie sous le nom de « LLM-as-a-judge », doublée d'un contrôle automatique des faits.

Slide 16 — Résultats ■ ~2 min [LE moment clé — prends ton temps]

Et voici les résultats. [laisser le graphique s'afficher, pointer] Je retiens trois enseignements. Premièrement, l'**effet causal du RAG est net** : à prompt identique, l'ajout du RAG fait passer l'exactitude de 0,66 à 0,98. Quasiment du simple au double. Deuxièmement, sur le système réel, entre la version 1 et la version 2, le nombre d'hallucinations chute de **neuf à une** — et la version 2 est même plus rapide et moins chère, grâce à la mise en cache. Et troisièmement, le résultat le plus intéressant : la meilleure configuration n'est pas celle qui est livrée. C'est celle où le prompt est complètement **dépouillé**, et où tout le savoir passe par le RAG : on atteint 0,98 d'exactitude, et **zéro hallucination**. [pause] Ça m'amène à une recommandation, mais avant, je veux être honnête sur ce que le RAG ne résout pas.

Slide 17 — Le RAG ne règle pas tout ■ ~1 min 30

Parce que le RAG n'est pas magique, et c'est important de le dire. Trois nuances. D'abord, baker le savoir dans le prompt peut activement **tromper** le modèle : dans la version 1, des règles se contaminent entre elles — une règle propre à l'E85 déborde sur la reprogrammation, et le bot affirme à tort qu'un diesel n'est pas éligible. C'est pour ça qu'il y a neuf hallucinations dans cette version. Ensuite, sur les **codes défauts OBD**, le RAG n'apporte rien : les deux conditions sont parfaites, parce que c'est une connaissance générale, déjà maîtrisée par le modèle. Donc la valeur du RAG se concentre sur le **savoir métier spécifique et qui change** — les prix, les compatibilités — pas sur les connaissances techniques générales. En revanche, point positif : les ruptures de conversation que je vous ai montrées, elles, sont bien corrigées.

Slide 18 — Recommandation d'ingénierie ■ ~1 min

D'où ma recommandation, qui est concrète et actionnable. Puisque baker les faits dans le prompt finit par interférer avec le RAG, il faut **sortir l'intégralité des faits du prompt** — prix, compatibilités — et tout faire passer par la base RAG. C'est la configuration qui obtient le meilleur résultat : 0,98 d'exactitude, zéro hallucination, et en prime, c'est **moins cher et plus simple à maintenir**, puisqu'un tarif se met à jour en éditant un seul fichier. Donc plus exact, moins cher, plus maintenable : c'est un résultat d'ingénierie directement applicable en production.

Slide 19 — Limites & perspectives ■ ~1 min

Quelles limites ? Je les assume. Mon évaluation est **hors-ligne** : tant que le bot n'est pas déployé, je ne mesure pas la satisfaction client réelle. Mon juge automatique reste faillible — d'ailleurs, j'ai changé de juge en cours de route parce que le premier se trompait. Et la qualité des réponses reste **bornée par celle des sources** : une base fausse donnerait des réponses fausses. Côté perspectives, trois pistes : des agents capables de produire un devis ou de réserver un créneau de bout en bout ; une extension multilingue, français-arabe, pour la clientèle locale ; et un fine-tuning léger, non pas pour les faits, mais pour affiner le **ton** de la marque.

Slide 20 — Conclusion & apports ■ ~1 min

Pour conclure. Je suis parti d'un besoin concret — décharger une PME de demandes répétitives — avec une exigence non négociable : ne jamais mentir au client. Et j'ai montré, **chiffres à l'appui**, que le RAG rend cet assistant à la fois fiable et facile à maintenir. Sur le plan personnel, cette mission m'a fait monter en compétences sur la conception d'un pipeline RAG de bout en bout, le déploiement, et surtout sur une démarche d'**évaluation rigoureuse**. Et sur le plan transverse, j'ai conduit ce projet en autonomie, de la première version jusqu'à la mise en production imminente. Ces acquis couvrent les compétences du référentiel, de l'analyse du besoin jusqu'à l'exploitation de la solution.

Slide 21 — Merci ■ ~10 s

Je vous remercie de votre attention, et je suis à votre disposition pour vos questions. [sourire, poser le regard sur le jury]

Repères pendant que tu parles

- Si tu sens que tu accélères (stress) → **ralentis** sur les slides 16, 17, 18.
- Garde une montre/un œil sur l'heure : à mi-présentation (~10 min) tu dois être vers le **slide 13**.
- Transitions à ne pas oublier : « *ce qui m'amène à...* », « *laissez-moi vous montrer...* », « *d'où ma recommandation...* ».
- Termine toujours une slide par une phrase qui **ouvre la suivante**.